

```
"""
```

```
|| CARRY · ROLL-DOWN MONITOR — IRS / XCCY / FX ||
```

```
Run: python carry_dashboard.py
```

```
Keys: R = refresh | Q = quit | 1-7 = filter currency | 0 = all
```

```
"""
```

```
import matplotlib
matplotlib.use("Agg") # headless / file output
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import FancyBboxPatch, Rectangle
from matplotlib.collections import LineCollection
import matplotlib.path_effects as pe
import numpy as np
import pandas as pd
from datetime import datetime
import random
import io, os, sys

# -----
# MARKET DATA (swap mock curves)
# -----

CURRENCIES = ["USD", "EUR", "GBP", "JPY", "CHF", "AUD", "CAD"]
TENORS = ["1Y", "2Y", "3Y", "5Y", "7Y", "10Y"]
TENOR_YEARS = [1, 2, 3, 5, 7, 10]
FLAGS = {"USD": "🇺🇸", "EUR": "🇪🇺", "GBP": "🇬🇧", "JPY": "🇯🇵", "CHF": "🇨🇭", "AUD": "🇦🇺"}

BASE_CURVES = {
    "USD": [5.30, 4.90, 4.70, 4.50, 4.45, 4.40],
    "EUR": [3.80, 3.50, 3.30, 3.10, 3.05, 3.00],
    "GBP": [5.10, 4.75, 4.55, 4.35, 4.30, 4.25],
    "JPY": [0.10, 0.25, 0.40, 0.65, 0.80, 1.00],
    "CHF": [1.20, 1.10, 1.05, 1.00, 0.98, 0.95],
    "AUD": [4.50, 4.30, 4.20, 4.10, 4.05, 4.00],
    "CAD": [5.00, 4.70, 4.55, 4.35, 4.30, 4.25],
}

FUNDING_SPREAD = {
    "USD": 0.05, "EUR": 0.03, "GBP": 0.04,
    "JPY": 0.02, "CHF": 0.02, "AUD": 0.05, "CAD": 0.04,
}
```



```

        else "AVOID")
    rows.append(dict(
        ccy=ccy, tenor=TENORS[i], rate=rate,
        funding=fund, carry=carry, rolldown=rolldown,
        total=total, dv01=dv01, breakeven=be, signal=signal
    ))
    return pd.DataFrame(rows).sort_values("total", ascending=False).reset_index(d

# -----
#   DRAWING HELPERS
# -----
def bp(v):
    return f"{v*100:+.1f}bp"

def pct(v):
    return f"{v*100:.2f}%"

def sig_color(s):
    return {"STRONG BUY": GREEN, "BUY": "#80d090",
            "NEUTRAL": YELLOW, "AVOID": RED}.get(s, TXT_MID)

def draw_rounded_rect(ax, x, y, w, h, color, alpha=1.0, radius=0.003, lw=0):
    fancy = FancyBboxPatch((x, y), w, h,
        boxstyle=f"round,pad=0",
        facecolor=color, edgecolor="none",
        alpha=alpha, transform=ax.transAxes, clip_on=False, linewidth=lw)
    ax.add_patch(fancy)

def draw_bar_inline(ax, x, y, value, max_val, width=0.06, height=0.012, color=GRE
    """Draw a tiny horizontal bar in axes-fraction coords."""
    frac = min(abs(value) / max(abs(max_val), 1e-6), 1.0)
    bg = Rectangle((x, y - height/2), width, height,
        transform=ax.transAxes, color=BORDER, clip_on=False)
    ax.add_patch(bg)
    fill = Rectangle((x, y - height/2), width * frac, height,
        transform=ax.transAxes, color=color, clip_on=False, alpha=0.8)
    ax.add_patch(fill)

# -----
#   MAIN RENDER
# -----
def render_dashboard(curves=None, horizon=0.5, history=None, iteration=0):
    if curves is None:
        curves = get_live_curves()

    df = compute_metrics(curves, horizon)

```

```

# rolling history for sparklines
if history is None:
    history = {ccy: [BASE_CURVES[ccy][3]] for ccy in CURRENCIES}
for ccy in CURRENCIES:
    history[ccy].append(curves[ccy][3])
    if len(history[ccy]) > 24:
        history[ccy] = history[ccy][-24:]

fig = plt.figure(figsize=(20, 13), facecolor=BG_DARK)
fig.patch.set_facecolor(BG_DARK)

gs = gridspec.GridSpec(
    4, 7,
    figure=fig,
    hspace=0.38,
    wspace=0.22,
    top=0.93, bottom=0.04,
    left=0.01, right=0.99
)

# — HEADER —————
ax_hdr = fig.add_subplot(gs[0, :])
ax_hdr.set_facecolor(BG_DARK)
ax_hdr.set_xlim(0, 1); ax_hdr.set_ylim(0, 1)
ax_hdr.axis("off")

# top accent line
ax_hdr.axhline(1.0, color=CYAN, linewidth=2, alpha=0.6)

ax_hdr.text(0.012, 0.72, "CARRY · ROLL-DOWN MONITOR",
            transform=ax_hdr.transAxes, fontsize=17, fontweight="bold",
            color=CYAN, fontfamily="monospace",
            path_effects=[pe.withStroke(linewidth=4, foreground=BG_DARK)])

ax_hdr.text(0.012, 0.22, "IRS · XCCY · FX SWAP | GLOBAL RATES DESK",
            transform=ax_hdr.transAxes, fontsize=9,
            color=TXT_DIM, fontfamily="monospace")

ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
ax_hdr.text(0.988, 0.72, f"[LIVE] {ts}",
            transform=ax_hdr.transAxes, fontsize=9, ha="right",
            color=GREEN, fontfamily="monospace")

horizon_lbl = {0.25:"3M", 0.5:"6M", 1.0:"1Y"}.get(horizon, f"{int(horizon*12)}")
ax_hdr.text(0.988, 0.22, f"HORIZON: {horizon_lbl} | ITER #{iteration:04d}",
            transform=ax_hdr.transAxes, fontsize=9, ha="right",
            color=TXT_DIM, fontfamily="monospace")

```

```

# — KPI CARDS —————
pos_count = (df["total"] > 0).sum()
avg_carry = df["carry"].mean()
avg_roll  = df["rolldown"].mean()
best      = df.iloc[0]

kpis = [
    ("AVG CARRY",      bp(avg_carry),   f"horizon {horizon_lbl}",   GREEN if
    ("AVG ROLL-DOWN", bp(avg_roll),     f"horizon {horizon_lbl}",   CYAN if
    ("BEST PAIR",      f"{best['ccy']} {best['tenor']}", f"total {bp(best['tot
    ("POSITIVE OPP.", f"{pos_count}/{len(df)}", "pairs with C+R > 0", PURPLE)
]

ax_kpi = fig.add_subplot(gs[1, :])
ax_kpi.set_facecolor(BG_DARK)
ax_kpi.set_xlim(0, 1); ax_kpi.set_ylim(0, 1)
ax_kpi.axis("off")

card_w = 0.23
for i, (label, val, sub, color) in enumerate(kpis):
    x0 = 0.005 + i * (card_w + 0.007)
    # card background
    rect = FancyBboxPatch((x0, 0.04), card_w, 0.88,
        boxstyle="round,pad=0.01",
        facecolor=BG_PANEL, edgecolor=BORDER,
        linewidth=0.8, transform=ax_kpi.transAxes, clip_on=False)
    ax_kpi.add_patch(rect)
    # top accent
    ax_kpi.plot([x0, x0 + card_w], [0.92, 0.92],
        color=color, linewidth=2.5, alpha=0.8,
        transform=ax_kpi.transAxes, clip_on=False)
    # label
    ax_kpi.text(x0 + 0.012, 0.73, label,
        transform=ax_kpi.transAxes, fontsize=8.5,
        color=TXT_DIM, fontfamily="monospace")
    # value
    ax_kpi.text(x0 + 0.012, 0.38, val,
        transform=ax_kpi.transAxes, fontsize=18,
        color=color, fontfamily="monospace", fontweight="bold")
    # sub
    ax_kpi.text(x0 + 0.012, 0.10, sub,
        transform=ax_kpi.transAxes, fontsize=8,
        color=TXT_MID, fontfamily="monospace")

# — MAIN TABLE —————
ax_tbl = fig.add_subplot(gs[2, :])

```

```

ax_tbl.set_facecolor(BG_DARK)
ax_tbl.set_xlim(0, 1); ax_tbl.set_ylim(0, 1)
ax_tbl.axis("off")

cols    = ["CCY", "TENOR", "SWAP RATE", "FUNDING", "CARRY", "ROLL-DOWN", "TOTAL C+R"]
col_x   = [0.00, 0.07, 0.14, 0.22, 0.30, 0.38, 0.47,
max_rows = 14

# header row
ax_tbl.axhline(0.97, color=BORDER, linewidth=0.6)
for j, (ch, cx) in enumerate(zip(cols, col_x)):
    ax_tbl.text(cx + 0.002, 0.988, ch,
                transform=ax_tbl.transAxes, fontsize=7.5,
                color=TXT_DIM, fontfamily="monospace", fontweight="bold",
                va="top")
ax_tbl.axhline(0.965, color=CYAN, linewidth=0.8, alpha=0.4)

row_h   = 0.062
n_show  = min(max_rows, len(df))

max_total = df["total"].abs().max() or 1

for idx in range(n_show):
    row   = df.iloc[idx]
    y0    = 0.955 - (idx + 1) * row_h
    yc    = y0 + row_h * 0.38    # text center

    # alternating bg
    bg_col = BG_ROW_A if idx % 2 == 0 else BG_ROW_B
    rect = Rectangle((0, y0), 1.0, row_h,
                    transform=ax_tbl.transAxes,
                    facecolor=bg_col, edgecolor="none", clip_on=False)
    ax_tbl.add_patch(rect)

    total_c = GREEN if row["total"] > 0.003 else ("80d090" if row["total"] >
    carry_c = GREEN if row["carry"] > 0 else RED
    roll_c  = CYAN if row["rolldown"] > 0 else RED
    ccy_col = CCY_COLORS.get(row["ccy"], TXT_MAIN)

    def tt(x, txt, color=TXT_MAIN, fs=8.5, bold=False):
        ax_tbl.text(x + 0.002, yc, txt,
                    transform=ax_tbl.transAxes, fontsize=fs,
                    color=color, fontfamily="monospace", va="center",
                    fontweight="bold" if bold else "normal")

    tt(col_x[0], row["ccy"], ccy_col, bold=True)
    tt(col_x[1], row["tenor"], TXT_MID)

```

```

tt(col_x[2], pct(row["rate"]),    TXT_MAIN)
tt(col_x[3], pct(row["funding"]), TXT_DIM)
tt(col_x[4], bp(row["carry"]),    carry_c, bold=True)
tt(col_x[5], bp(row["rolldown"]), roll_c, bold=True)

# Total with inline bar
tt(col_x[6], bp(row["total"]),    total_c, bold=True)
draw_bar_inline(ax_tbl, col_x[6] + 0.055, yc,
                row["total"], max_total, width=0.04, height=0.018,
                color=total_c)

tt(col_x[7], f"${row['dv01']}:.0f)", TXT_DIM, fs=8)
tt(col_x[8], f"{row['breakeven']*100:.1f}", TXT_MID, fs=8)
tt(col_x[9], row["signal"], sig_color(row["signal"]), bold=True, fs=8)

ax_tbl.axhline(y0, color=BORDER, linewidth=0.3, alpha=0.5)

# — CURVE PANELS (bottom row) —————
for ci, ccy in enumerate(CURRENCIES):
    ax_c = fig.add_subplot(gs[3, ci])
    ax_c.set_facecolor(BG_PANEL)
    for sp in ax_c.spines.values():
        sp.set_color(BORDER); sp.set_linewidth(0.6)

    c_vals = [v * 100 for v in curves[ccy]] # in %
    col = CCY_COLORS[ccy]

    # shaded area under curve
    ax_c.fill_between(TENOR_YEARS, c_vals,
                      min(c_vals) - 0.1,
                      color=col, alpha=0.08)
    ax_c.plot(TENOR_YEARS, c_vals,
              color=col, linewidth=1.8, solid_capstyle="round")
    ax_c.scatter(TENOR_YEARS, c_vals,
                 color=col, s=18, zorder=5)

    # sparkline of 5Y history (top-right corner overlay)
    hist = history[ccy]
    if len(hist) > 2:
        hy = np.array(hist) * 100
        hx = np.linspace(0.55, 0.98, len(hy))
        # normalise to small box
        hy_n = (hy - hy.min()) / (np.ptp(hy) + 1e-6)
        hy_n = hy_n * 0.3 + 0.62
        ax_c.plot(hx, hy_n, color=col, linewidth=0.8, alpha=0.5,
                  transform=ax_c.transAxes)

```

```

ax_c.set_facecolor(BG_PANEL)
ax_c.tick_params(colors=TXT_DIM, labels=6.5)
ax_c.set_xticks([1, 3, 5, 10])
ax_c.set_xticklabels(["1Y", "3Y", "5Y", "10Y"], fontsize=6, color=TXT_DIM,
                      fontfamily="monospace")
ax_c.yaxis.set_tick_params(labels=6, colors=TXT_DIM)
ax_c.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.1f}%"))
ax_c.set_xlim(0.5, 11); ax_c.grid(False)

# currency label
ax_c.set_title(f"{ccy} {pct(curves[ccy][3])}",
              fontsize=9, fontweight="bold", color=col,
              fontfamily="monospace", pad=4)

# delta 5Y vs base
delta = (curves[ccy][3] - BASE_CURVES[ccy][3]) * 10000
d_col = GREEN if delta >= 0 else RED
ax_c.text(0.97, 0.07, f"{delta:+.1f}bp",
         transform=ax_c.transAxes, fontsize=7, ha="right",
         color=d_col, fontfamily="monospace")

# — FOOTER —
fig.text(0.5, 0.013,
        "CARRY = (SwapRate - FundingRate) × Horizon · ROLL-DOWN = (Rate[T] - "
        "SIGNAL: >40bp STRONG BUY | >20bp BUY | >0 NEUTRAL | <0 AVOID · "
        ha="center", fontsize=7.5, color=TXT_DIM, fontfamily="monospace")

return fig, curves, history

# —————
# EXPORT SNAPSHOTS
# —————
def export_snapshots(n=3, horizon=0.5):
    """Generate n snapshots simulating market moves."""
    curves = get_live_curves()
    history = {ccy: [BASE_CURVES[ccy][3]] for ccy in CURRENCIES}
    paths = []

    for i in range(n):
        curves = get_live_curves()
        fig, curves, history = render_dashboard(
            curves=curves, horizon=horizon,
            history=history, iteration=i + 1
        )
        path = f"/mnt/user-data/outputs/carry_dashboard_{i+1:02d}.png"
        fig.savefig(path, dpi=130, bbox_inches="tight",

```

```

        facecolor=BG_DARK, edgecolor="none")
plt.close(fig)
paths.append(path)
print(f" ✓ snapshot {i+1}/{n} → {path}")

return paths

if __name__ == "__main__":
    print("\n CARRY · ROLL-DOWN MONITOR – generating snapshots...\n")
    paths = export_snapshots(n=3, horizon=0.5)
    print(f"\n Done. {len(paths)} images saved.\n")

```